

Security and Functional Safety for AI in Embedded Automotive System—A Tutorial

Yi Wang¹, Senior Member, IEEE, Jing Xiao, Zhengzhe Wei, Member, IEEE,
Yuanjin Zheng², Senior Member, IEEE, Kea-Tiong Tang³, Senior Member, IEEE,
and Chip Hong Chang⁴, Fellow, IEEE

Abstract—The tutorial explores key security and functional safety challenges for Artificial Intelligence (AI) in embedded automotive systems, including aspects from adversarial attacks, long life cycles of products, and limited energy resources of automotive platforms within safety-critical environments in diverse use cases. It provides a set of recommendations for how the security and safety engineering of machine learning can address these challenges. It also provides an overview of contemporary security and functional safety engineering practices, encompassing up-to-date legislative and technical prerequisites. Finally, we identify the role of AI edge processing in enhancing security and functional safety within embedded automotive systems.

Index Terms—ML, functional safety, security, AI edge processor, adversarial attacks.

I. INTRODUCTION

THE DEVELOPMENT of Artificial Intelligence (AI), especially machine learning (ML), in the automotive sector is one of the biggest drivers of autonomous and intelligent transportation systems, for example in L4 and L5 automation, according to SAE J3061 [1]. ML, a branch of AI, is the process of optimizing model parameters through computational techniques, such that the model's behavior reflects the data or experience [2]. Massive data growth, higher connectivity on 5G/6G, and faster computing power have increased the capabilities of AI and ML processes. While it is imperative to provide efficient AI models to assist autonomous driving technology, the safety, confidentiality, integrity, and availability of AI models must also be considered at design time so that the deployed models are robust against attacks.

Manuscript received 1 September 2023; revised 27 October 2023; accepted 13 November 2023. Date of publication 22 November 2023; date of current version 5 March 2024. This brief was recommended by Associate Editor Y. Ha. (Corresponding author: Yi Wang.)

Yi Wang is with the Automotive Technology, Software and Central Technologies, Continental AG, 30165 Hanover, Germany (e-mail: estelle.wang@continental.com).

Jing Xiao is with the Continental Autonomous Mobility Germany GmbH, ENP Tech Office, Continental AG, 30165 Hanover, Germany (e-mail: jing.xiao@continental.com).

Zhengzhe Wei, Yuanjin Zheng, and Chip Hong Chang are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore (e-mail: zhengzhe001@e.ntu.edu.sg; YJZHENG@ntu.edu.sg; ECHChang@ntu.edu.sg).

Kea-Tiong Tang is with the Department of Electrical Engineering, National Taiwan University, Taipei City 10617, Taiwan (e-mail: kttang@mx.ntu.edu.tw).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSII.2023.3334273>.

Digital Object Identifier 10.1109/TCSII.2023.3334273

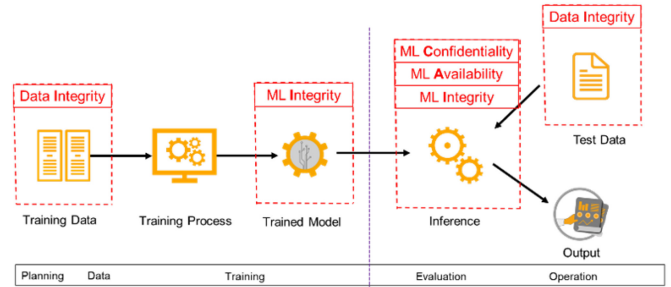


Fig. 1. Confidentiality, Integrity, Availability (CIA) in the product life cycle of ML.

In particular, the security assets of AI-assisted applications should be scrutinized against ML model attributes to prevent the latter from being manipulated and exploited to cause functional and safety issues when these models are integrated.

The security of ML is to ensure resilience to attacks and risk management throughout the entire life cycle (planning, data, training, evaluation, and operations phases) of ML. Adversarial attacks on ML include physical attacks, boundary attacks, etc. These attacks can be cataloged into three types according to their impact on security attributes: Confidentiality, Integrity, and Availability (CIA). Risk management is considered and implemented throughout the life cycle of ML, as shown in Fig. 1.

Safety engineering for ML is about managing potential technical risks that may arise from ML. Before ML-based software finds its way into safety-critical applications, such as autonomous driving functions, users of these systems must be assured of highly reliable operation.

Fig. 2 shows examples of safety issues in ML models (e.g., Neural Network (NN) design faults). Those faults will introduce errors in the out-of-NN models and lead to the insufficiency inherent in the ML function, resulting in an accident.

Potential faults introduced during the design of NN include, for example, that the NN architecture is not correctly specified, the training data set does not model the intended goal, and the implementation of the network prior to training is incorrect due to, for example, errors in the initial number of neurons, connection matrices, growing/learning functions, or activation functions [3]. We summarize the challenges involved in the safety assurance of ML in embedded automotive systems from three viewpoints: (1) From an ML technology viewpoint,

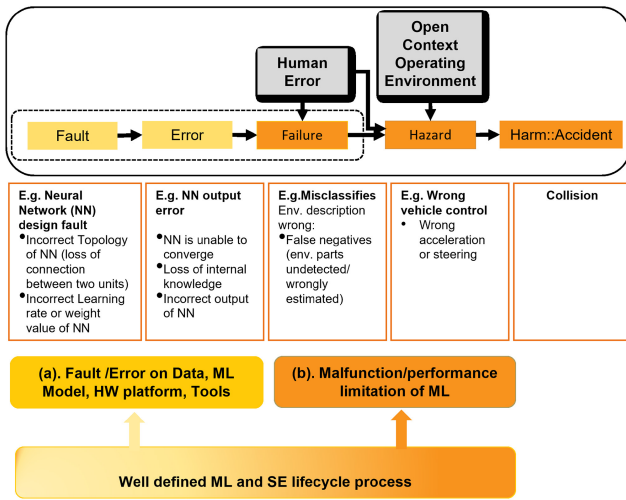


Fig. 2. Examples on ML Chain of Failure Events.

to identify safety issues on the data set and ML model; (2) From the safety process viewpoint, to specify a standardized safety lifecycle of ML, which aims to prevent systematic faults during ML design and development; (3) From the system safety viewpoint, to identify some architectural considerations related to ML technologies and ensure safety concepts at the system design level. A system safety argument strategy must be used to explain how and the extent to which the generated evidence supports the relevant ML safety issues.

AI edge processing ensures security and functional safety in embedded automotive systems. Here are some reasons why it is needed:

- **Real-time Decision Making**: Edge processing allows AI algorithms to run directly on the embedded automotive system, enabling real-time decision-making capabilities.
- **Reduced Latency**: By processing AI algorithms locally on the edge device, the need to transmit data to a centralized server or cloud for processing is eliminated.
- **Enhanced Privacy and Data Security**: Edge processing ensures that sensitive data collected by the embedded automotive system remains on the device, reducing the risks associated with transmitting data to external servers.
- **Robustness against Network Disruptions**: Embedded automotive systems often operate in environments with limited or intermittent network connectivity. By leveraging edge processing, these systems can continue functioning autonomously even when network connectivity is disrupted.
- **Reducing Dependence on External Infrastructure**: Edge processing reduces the reliance on external infrastructure, such as cloud servers, for processing AI algorithms.
- **Compliance with Safety Standards**: Automotive safety standards, such as ISO26262 [4] and ISO21448 [5], require robust safety measures for embedded systems. By incorporating AI edge processing, automotive systems can meet the necessary safety requirements by reducing dependencies on external factors and ensuring real-time decision-making capabilities.

Overall, AI edge processing in embedded automotive systems is essential for reliability, security, and functional safety. Reliability focuses on consistent performance, while security is

concerned with protecting against cyber threats, and functional safety emphasizes safety-critical operation. In complex systems, all three concepts often overlap and need to be considered together to create robust, secure, and safe solutions to ensure the integrity, availability, and dependability of systems.

In this tutorial, Section II provides up-to-date regulations and standards requirements for security and functional safety related to the automotive industry. An overview of existing works related to security and functional safety for ML is listed in Section III. Considering specific automotive system requirements, Section IV presents the challenges and opportunities for the role of AI edge processing in the future. Section V draws the conclusion.

II. UP-TO-DATE REGULATIONS AND STANDARDS

ISO/IEC TR 24368 [6] focuses on the overview of trustworthiness in artificial intelligence, which outlines the principles of responsibility, accountability, governance, safety, security, privacy, bias, unpredictability, and opacity. Trustworthiness is the ability to meet stakeholders' expectations verifiably. Security and safety assessments are critical elements for ML trustworthiness analysis. They are essential prerequisites in developing any system with embedded ML and are preliminary steps and integral processes toward approving such innovative solutions. The automotive domain will follow the minimum-security protection principle. Since UNECE R155 [7] takes effectiveness, this is a compulsory regulation to quantify the minimum security requirements in Annex 5. The Cybersecurity Assurance Level (CAL), defined by ISO/SAE 21434 [8], is an additional reference for security acceptance criteria. In addition, NIST also provided the taxonomy arrangements of different types of attacks, defenses, and consequences, in which the terminology defines critical terms related to the security of ML in AI systems [9].

For ML-based systems, the safety requirements at the system level are generated from safety analysis according to safety references, including ISO 26262 Functional Safety [4] and ISO 21448 Safety of the Intended Functionality (SOTIF) [5]. Risk analysis such as Hazard Analysis and Risk Assessment (HARA), Systems Theoretic Process Analysis (STPA), and safety in-use analysis are the main methods used to identify and assess associated hazards and risks at the system level. Operational concepts, system and functional specifications, HARA, and system safety requirements are the main inputs to derive safety requirements at the ML-based subsystem or component level. The specification is ISO PAS 8800. Functional safety and AI are currently under development to address a broad scope of AI/ML applications and technologies within road vehicles for safety-relevant automated driving tasks. It will describe the general principles to be fulfilled when arguing the safety of AI/ML and will include a reference safety life cycle for AI/ML [10].

III. SECURITY/SAFETY ISSUES AND MITIGATION APPROACHES FOR ML

From a security point of view, the security of ML is to ensure that the AI model can defend against adversarial attacks. In addition, how to meet the minimum security

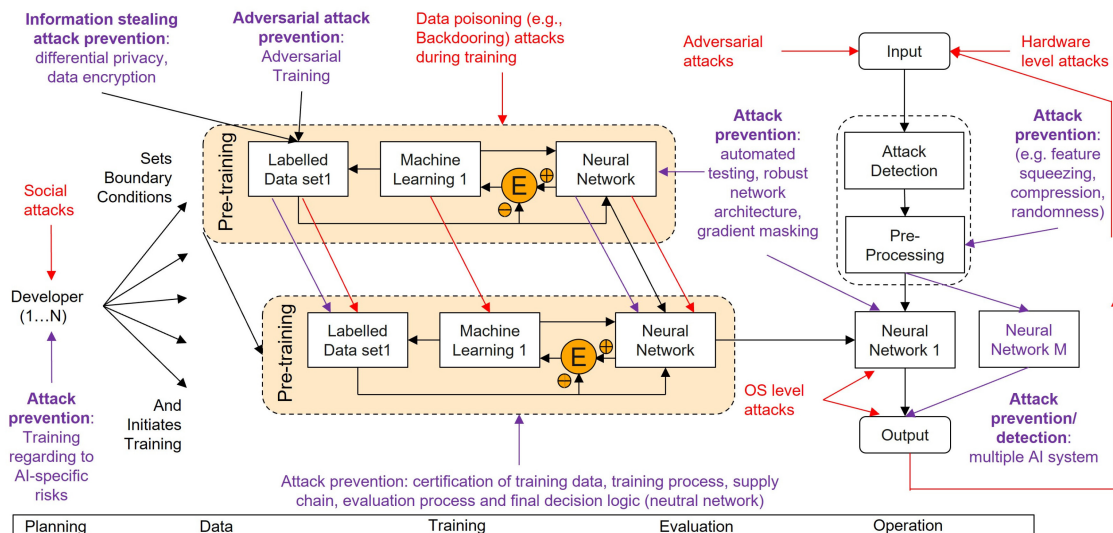


Fig. 3. Potential attacks and prevention for machine learning along the life cycles [11].

TABLE I
EXISTING ATTACKS OF ML ACCORDING TO SECURITY ATTRIBUTES

Security Attributes	Short Names	Attacks	Detection	Prevention	Descriptions
Confidentiality	Model Inversion	[13] [14] [15]	-	[16] [17]	The attacker recovers the secret features used in the model through careful queries.
Confidentiality	Membership Interface	[18] [19]	-	[20]	Attackers can infer if a given data record was part of the model's training dataset or not.
Confidentiality	Malicious ML provider recovering	[21]	-	-	A malicious ML provider can query the model used by the customer and recover the customer's training data.
Confidentiality	Module stealing/extraction	[22] [23]	-	[23][24] [25] [26]	Attack is able to recover the model through carefully crafted queries.
Integrity	Poisoning attacks	[27] [28]	[29]	[30] [31][32]	Attacker contaminates the training phase of ML systems to get the intended result.
Integrity	Attacking the ML Supply Chain	[33] [34]	-	-	The attack compromises the ML models as they are being downloaded for use.
Integrity	Perturbation/Adversarial attacks	[35] [36] [37] [38] [39] [40] [41] [42] [43]	[44] [45]	[46] [47]	The attacker modifies the query to get appropriate response.
Integrity	Adversarial examples in the physical domain	[48] [49]	-	-	Attacks bring adversarial examples into the physical domain to subvert the ML system. (e.g., 3D printing special eyewear to fool the facial recognition system).
Confidentiality, Integrity	Backdoor ML	[50] [51] [52]	[53]	-	Malicious ML provider backdoors algorithm to activate with a specific trigger
Confidentiality, Integrity	Reprogramming ML Systems	[54] [55]	-	-	Re-purpose the ML system to perform an activity it was not programmed for.
Availability	Exploit software dependencies	[55]	-	[56]	Attackers use traditional software exploits like buffer overflow to confuse or control ML systems.

requirements in industrial practice is popularly discussed in up-to-date regulations and standards. Fig. 3 lists the attacks and prevention along the life cycle of machine learning in the phases of planning, data, training, evaluation, and operation [11]. Table I summarizes the above attacks [12], which can be cataloged according to the compromise of the security attributes. Specifically, training data is a critical component in the development of neural networks. Its key aspects of the quality of training data are: representativeness, accuracy, completeness, consistency, relevance, ethical considerations, data augmentation, validation and testing sets, adaptability, and privacy and security.

From a safety point of view, safety issues in ML refer to foundational issues that have the potential to impact the overall safety of a system detrimentally. They can serve as the fundamental causes of functional or performance limitations

for ML functions. According to ISO 21448 [5], functional insufficiencies are inherent in the system, possibly leading to hazards. Such an insufficiency can appear, e.g., in the form of a performance limitation leading to an incomplete or wrong perception of the environment [57]. Safety issues can also refer to a situation where a system, particularly one involving deep learning or complex algorithms, exhibits behavior that is difficult to understand or predict, which makes it hard to argue and assess system safety. Safety engineering for an ML-based system requires a comprehensive, evidence-based safety argument to show that all safety issues are identified, analyzed, and mitigated. Table II provides an overview of safety issues, their impacts at the system level, and recommended mitigation approaches. The content is mainly based on the publications [3], [10], [57], [58], [59], [60] but does not

TABLE II
SAFETY ISSUES FOR ML AND RECOMMENDED MITIGATION APPROACHES, SUMMARIZED FROM [3], [10], [57], [58], [59], [60]

Categories	Safety issues	Short description of system impacts	Recommended mitigation approaches
Data	Data distribution are not valid approximations of the real world	Incomplete or wrong operational design domain coverage can lead to the system making incorrect decisions when faced with situations that fall outside its training data.	Collect diverse and representative data, continuously monitor data distribution shifts, and use techniques like domain adaptation and concept drift detection to ensure that the ML system performs reliably and safely in various real-world scenarios.
Data	Inadequate separation of test and training data	Problem of data leakage. This can result in overly optimistic performance estimates, where the model appears to perform well on the test data because it has seen some of that test data during training.	Ensure a clear separation between the training, validation, and testing data. Techniques like cross-validation and holding out a dedicated test set that the model has yet to be exposed to during training can help properly evaluate the system's performance and ensure its generalization capabilities to new data.
Data	Dependence on labeling quality	Insufficient labeling quality of data used for training or testing. For example, poor labeling can result in a model learning incorrect associations between inputs and labels, leading to degraded performance when deployed in real-world scenarios.	Ensure the reliability of labeled data. Ensure high-quality, accurate, and consistent training and testing data labeling. This might involve employing expert annotators, implementing rigorous quality control processes, and considering techniques like adversarial testing to identify potential issues with labeling quality.
Data	Distributional shift over time	Domain or concept drift over time during operation (e.g., sensor degradation), this can lead to situations where the system needs to make correct predictions or decisions due to encountering inputs significantly different from those seen during training.	Ongoing monitoring of the system's performance in the real-world environment is crucial. Techniques such as domain adaptation, continuous learning, and retraining with updated data can help systems remain effective and safe despite changes in data distribution.
Data	Unknown behavior in rare critical situations	Systems might struggle to respond appropriately, and failure to handle rare critical situations effectively can lead to accidents, harm, or unintended consequences.	Design ML systems to account for various scenarios, including rare and critical situations. Techniques like simulation, adversarial testing, and reinforcement learning from human feedback can help AI systems develop robust responses in scenarios that may need to be better represented in the training data. Regular testing and validation against rare critical situations are crucial to ensuring the safety and reliability of systems in the real world.
NN	Lack of temporal stability	For example, a lack of temporal stability can erode users' trust in the system as its performance becomes unpredictable and difficult to understand.	Implement mechanisms for continuous monitoring and evaluation of the system's performance over time. Regular retraining with updated data and feedback loops can help maintain the system's stability and ensure it performs reliably as conditions change.
NN	Incomprehensible behavior	Incomprehensible behavior makes it difficult to diagnose issues, troubleshoot errors, or understand why a particular decision was made, hindering the ability to improve and maintain the system. It also becomes challenging to trust their outputs.	Develop AI systems that are interpretable and explainable. These systems provide insights into how decisions are made, which features are important, and why specific outcomes are generated. Techniques like model interpretability, attention mechanisms, and visualization of internal processes help make AI systems more transparent and understandable, enhancing their trustworthiness and facilitating their integration into various applications.
Safety metric	Lack of quantitative safety metrics for the identified safety issues	For example, this will lead to an incomplete safety argument and influence the end user's trust in the system	To measure if a safety concern was mitigated sufficiently, safety metrics must be defined accordingly. For example, "mean average precision (mAP) under perturbation" will measure the performance of the perception function under various kinds of known perturbations (e.g., weather effects).
Computational resource	Computational resource limits are approached or exceeded	It may happen with growing NNs that add more neurons and connections over time. This may lead to system performance degrading, leading to slower response times, reduced accuracy, or even system failures.	Optimize AI algorithms for efficiency and scalability to ensure they can perform effectively within the available computational resources. Techniques like model pruning, quantization, and hardware acceleration can help manage resource consumption. Additionally, careful design and testing are crucial to prevent the AI system from operating in resource-constrained scenarios where its behavior could become unpredictable or unsafe.
Operational monitors	Inappropriate operational monitor function	Operational monitor function as either too restrictive or not restrictive enough. For example, when the monitor function is too restrictive, it might hinder the ML system's performance, leading to missed opportunities or unnecessary disruptions.	AI systems must have monitoring functions that are carefully calibrated based on a thorough understanding of the system's behavior, potential risks, and the specific operational environment. Regular testing, evaluation, and updates to the monitoring mechanism are necessary to ensure that it neither hampers the system's effectiveness nor compromises safety.
Process	Lack of evidence-based safety arguments	Due to the lack of a normalized safety lifecycle process for ML, documentation during the design and development of ML is missing	Standardization efforts are currently underway to fill the gap. The upcoming ISO 8800 "Functional Safety and AI" will propose a reference AI safety lifecycle process.

claim an exhaustive list. It aims to help ML developers and safety engineers better understand and manage safety issues.

IV. AI EDGE PROCESSING FOR SECURITY AND FUNCTIONAL SAFETY: CHALLENGE AND OPPORTUNITIES

The demand for low-power and low-latency real-time AI inference drives the adoption of application-specific circuits and systems that must be custom-designed and manufactured. Regarding the functional security and safety of AI systems, this section first highlights the challenges of trust in IC

manufacturing and usage. Next, referring to Fig. 4, the trade-offs of various edge computing implementations are discussed. Finally, emerging opportunities such as sensor fusion and bio-inspired algorithm-hardware co-design methods are introduced.

(1) IC design companies outsource fabrication to offshore foundries to offset the prohibitive cost of semiconductor fabrication and take advantage of economies of scale. However, such global collaboration also introduces a wide range of threats, including the injection of hardware trojans (HT) and the sale of ICs rejected by tests, compromising the safety

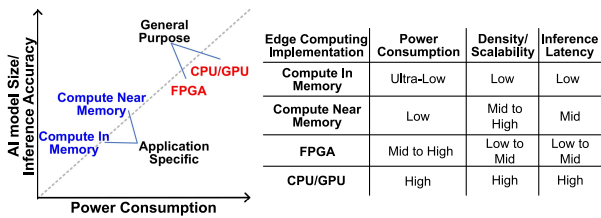


Fig. 4. Trade-offs of various edge computing hardware for implementing large AI models.

of the overall system. For instance, a hardware trojan is embedded into a hardware system without the authorization of the designer. The hardware trojan can be implemented as an additional circuit on the hardware accelerator, occupying an additional area [61]. In addition, the hardware trojan can be injected without incurring area overhead in an FPGA look up table (LUT) network [62].

In response to the hardware security vulnerabilities, ongoing studies reveal vulnerabilities of AI modules to develop detection methods [63], [64], [65], [66] for security risk mitigation. On the other hand, design-for-security architecture provides preventative measures to prevent counterfeiting, IP piracy, and the unauthorized sale of rejected ICs. For instance, Guin et al. [67] introduce an architecture that supports testing before activation with the correct key. An AES encryption block is embedded to increase the difficulty of the satisfiability (SAT) attack [68]. However, silicon area overhead must be taken into consideration while maintaining robust security.

(2) The accuracy and latency of AI inference contribute to AI systems' functional safety. However, edge implementations face the challenges of optimizing energy and area efficiency while utilizing large neural network models with high parameter counts. A typical von Neumann architecture, including the CPU and GPU, demands high power consumption due to frequent data movement across the memory and processing cores. To achieve high energy efficiency, custom hardware architectures, e.g., compute in memory (CIM) and compute near memory (CNM), have been proposed to reduce power consumption in various neuromorphic systems.

CIM minimizes power consumption by localizing runtime memory and performing computation directly within memory at the expense of low memory density [69], [70], [71]. For instance, to accommodate the need for both compute and data storage, a typical 6T SRAM is modified by adding an additional pair of transistors to decouple each SRAM cell sharing one bit-line, addressing the read-disturb issue during a compute cycle at the expense of reduced density [72]. Alternatively, novel non-volatile memory-based CIM solutions have also been proposed to increase density and reduce standby energy consumption. In design [73], RRAM is studied and evaluated. It was recognized that despite the improved density, RRAM suffers from reliability and endurance issues. For instance, it was shown that the system accuracy can degrade from 91.6% to 43% due to RRAM reliability soft errors. On the other hand, its endurance can be extended using algorithms demonstrated in [74]. CNM provides enhanced scalability while inevitably increasing latency. Recently, in addition to circuit-level exploration, an architectural-level

investigation has been carried out [75], implementing a heterogeneous architecture incorporating both CIM and CNM. The algorithm responsible for the system-level operation is partitioned into two parts that map to a CIM macro and a CNM macro, taking advantage of both macros' benefits.

(3) AI edge processing poses the challenge of developing tailored algorithms in collaboration with hardware design in the domain of safety-focused automotive applications.

Sensor fusion combines multiple sensors to enhance system safety and accuracy. For instance, a radar sensor and a LiDAR sensor are complementary. A radar sensor can provide a long detection range in various visibility conditions. However, radar data suffers from low resolution and lacks height information. On the other hand, LiDAR can provide detailed sensing data in the short range but cannot function under low-visibility conditions. However, the efficient fusion of both sensors is non-trivial, given the disparity between radar and LiDAR features. A prior study explored a bi-directional LiDAR-Radar fusion scheme to improve the accuracy of 3D object detection [76]. In [77], radar and LiDAR fusion are explored to provide early fusion and energy savings for edge processing.

AI edge processing requires a custom algorithm for hardware-algorithm co-design. For instance, accurate simultaneous localization, and mapping (SLAM) must be performed for robotic and automotive applications. Although various computational methods are available [78], [79], probabilistic SLAM, such as the extended Kalman filter (EKF), is not practical for edge implementation due to its extensive demand for large matrix computation, data pre-processing, and post-processing. The existing implementation of EKF SLAM [80] can only accelerate the EKF algorithm partially through an embedded matrix multiplication accelerator while still relying on the CPU to handle complex tasks such as matrix partition, trigonometry computation, and data flow management. An opportunity awaits for potential algorithm development to meet the stringent demand for edge processing. A work in [81] demonstrates a bio-inspired lightweight SLAM algorithm and its low-power edge hardware implementation. Yet, further work needs to be done to improve its robustness via sensor fusion and improve its accuracy and precision.

V. CONCLUSION

This tutorial delves into the significant challenges posed by security and functional safety in the context of Artificial Intelligence within embedded automotive systems. These challenges encompass a range of factors. The tutorial provides an overview and recommendations for adapting security and functional safety engineering to machine learning. Additionally, it provides an up-to-date overview of regulations and standards in security and functional safety engineering. Finally, the tutorial highlights that AI edge processing is indispensable in embedded automotive systems to bolster security and functional safety. It enables instantaneous decision-making, minimizes latency, fortifies data privacy and security, guarantees resilience in network disruptions, diminishes external infrastructure dependency, and facilitates compliance with vital safety standards.

REFERENCES

- [1] *Cybersecurity Guidebook for Cyber-Physical Vehicle Systems*, SAE Standard J3061, Jan. 2016.
- [2] *Information Technology—Artificial Intelligence—Artificial Intelligence Concepts and Terminology*, ISO/IEC Standard 22989, 2022.
- [3] B. J. Taylor, *Methods and Procedures for the Verification and Validation of Artificial Neural Networks*, Inst. Sci. Res., Farimont, WV, USA, 2010.
- [4] *Road Vehicles—Functional Safety*, ISO Standard 26262, 2018.
- [5] *Road Vehicles—Safety of the Intended Functionality*, ISO Standard 21448, 2022.
- [6] *Information Technology—Artificial Intelligence—Overview of Ethical and Societal Concerns*, ISO/IEC Standard AWI TR 24368, 2019.
- [7] “UN regulation No. 155—Cybersecurity and cybersecurity management system.” 2021. [Online]. Available: <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security>
- [8] *Road Vehicles—Cybersecurity Engineering*, ISO/SAE Standard 21434, 2021.
- [9] “A taxonomy and terminology of adversarial machine learning,” NIST, Gaithersburg, MD, USA, Rep. NISTIR 8269, 2019.
- [10] S. Burton, *AE EDGE Research Report: The Path to Safe Machine Learning for Automotive Applications*, SAE Int., Warrendale, PA, USA, 2023.
- [11] C. Berthoff et al. “Towards auditable AI systems current status and future directions.” Accessed: May 2021. [Online]. Available: <https://www.hhi.fraunhofer.de/fileadmin/Departments/AI/TechnologiesAndSolutions/AuditingAndCertificationOfAiSystems/2021-05-04-whitepaper-tuev-bsi-hhi-towards-auditable-ai-systems.pdf>
- [12] R. S. S. Kumar, J. Snover, D. O’Brien, K. Albert, and S. Viljoen. “Machine learning process documentation—DEVOPS vs MLOPS.” 2021. [Online]. Available: <https://docs.microsoft.com/en-us/security/engineering/threat-modeling-aiml>
- [13] M. Fredrikson, S. Jha, and T. Ristenpart, “Model inversion attacks that exploit confidence information and basic countermeasures,” in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Security*, 2015, pp. 1322–1333.
- [14] T. Zhu, D. Ye, S. Zhou, B. Liu, and W. Zhou, “Label-only model inversion attacks: Attack with the least information,” *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 991–1005, 2022.
- [15] S. Zhou, T. Zhu, D. Ye, X. Yu, and W. Zhou, “Boosting model inversion attacks with adversarial examples,” *IEEE Trans. Depend. Secure Comput.*, early access, Jun. 12, 2023, doi: [10.1109/TDSC.2023.3285015](https://doi.org/10.1109/TDSC.2023.3285015).
- [16] Y. Alufaisan, M. Kantarcioglu, and Y. Zhou, “Robust transparency against model inversion attacks,” *IEEE Trans. Depend. Secure Comput.*, vol. 18, no. 5, pp. 2061–2073, Sep/Oct. 2021.
- [17] X. Gong, Z. Wang, S. Li, Y. Chen, and Q. Wang, “A GAN-Based defense framework against model inversion attacks,” *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4475–4487, 2023.
- [18] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proc. IEEE Symp. Security Privacy*, 2017, pp. 3–18.
- [19] L. Liu, Y. Wang, G. Liu, K. Peng, and C. Wang, “Membership inference attacks against machine learning models via prediction sensitivity,” *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 3, pp. 2341–2347, May/Jun. 2023.
- [20] Y. Mao, W. Hong, B. Zhu, Z. Zhu, Y. Zhang, and S. Zhong, “Secure deep neural network models publishing against membership inference attacks via training task parallelism,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 3079–3091, Nov. 2022.
- [21] C. Song, T. Ristenpart, and V. Shmatikov, “Machine learning models that remember too much,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2017, pp. 587–601.
- [22] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing machine learning models via prediction APIs,” in *Proc. 25th USENIX Security Symp. (USENIX Security)*, 2016, pp. 601–618.
- [23] D. Oliynyk, R. Mayer, and A. Rauber, “I know what you trained last summer: A survey on stealing machine learning models and defences,” *ACM Comput. Surveys*, vol. 55, no. 324, pp. 1–41, 2023.
- [24] W. Jiang, H. Li, G. Xu, T. Zhang, and R. Lu, “A comprehensive defense framework against model extraction attacks,” *IEEE Trans. Depend. Secure Comput.*, early access, Mar. 24, 2023, doi: [10.1109/TDSC.2023.3261327](https://doi.org/10.1109/TDSC.2023.3261327).
- [25] T. Orekondy, B. Schiele, and M. Fritz, “Prediction poisoning: Towards defenses against DNN model stealing attacks,” 2020, *arXiv:1906.10908*.
- [26] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, “PRADA: Protecting against DNN model stealing attacks,” in *Proc. IEEE Eur. Symp. Security Privacy (EuroS&P)*, 2019, pp. 512–527.
- [27] G. Sun, Y. Cong, J. Dong, Q. Wang, L. Lyu, and J. Liu, “Data poisoning attacks on federated machine learning,” *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11365–11375, Jul. 2022.
- [28] K. Liu, B. Tan, R. Karri, and S. Garg, “Training data poisoning in ML-CAD: Backdooring DL-based lithographic hotspot detectors,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 6, pp. 1244–1257, Jun. 2021.
- [29] X. Li, Z. Qu, S. Zhao, B. Tang, Z. Lu, and Y. Li, “LoMar: A local defense against poisoning attack on federated learning,” *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 1, pp. 437–450, Jan./Feb. 2023.
- [30] K. Li, C. Baird, and D. Lin, “Defend data poisoning attacks on voice authentication,” *IEEE Trans. Depend. Secure Comput.*, early access, Jun. 26, 2023, doi: [10.1109/TDSC.2023.3289446](https://doi.org/10.1109/TDSC.2023.3289446).
- [31] J. Chen, X. Zhang, R. Zhang, C. Wang, and L. Liu, “De-Pois: An attack-agnostic defense against data poisoning attacks,” *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3412–3425, 2021.
- [32] Z. Wu et al., “Threshold switching memristor based radial based spiking neuron circuit for conversion based spiking neural networks adversarial attack improvement,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 8, no. 2, pp. 619–629, 2021.
- [33] D. Williams, C. Clark, R. McGahan, B. Potteiger, D. Cohen, and P. Musau, “Discovery of AI/ML supply chain vulnerabilities within automotive cyber-physical systems,” in *Proc. IEEE Int. Conf. Assured Autonomy (ICAA)*, 2022, pp. 93–96.
- [34] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Identifying vulnerabilities in the machine learning model supply chain,” 2019, *arXiv:1708.06733*.
- [35] Y. Dong et al., “Boosting adversarial attacks with momentum,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 9185–9193.
- [36] C. Guo, J. Gartner, Y. You, A. G. Wilson, and K. Weinberger, “Simple black-box adversarial attacks,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2484–2493.
- [37] N. Akhtar and A. Mian, “Threat of adversarial attacks on deep learning in computer vision: A survey,” *IEEE Access*, vol. 6, pp. 14410–14430, 2018.
- [38] C. Zhang, P. Benz, C. Lin, A. Karjauv, J. Wu, and I. S. Kweon, “A survey on universal adversarial attack,” in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 4687–4694.
- [39] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learn. Rep.*, 2014, pp. 1–9.
- [40] J. Hayes and G. Danezis, “Learning universal adversarial perturbations with generative models,” in *Proc. IEEE Security Privacy Workshops (SPW)*, 2017, pp. 43–49.
- [41] O. Poursaeed, I. Katsman, B. Gao, and S. J. Belongie, “Generative adversarial perturbations,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4422–4431.
- [42] T. Xiang, H. Liu, S. Guo, Y. Gan, and X. Liao, “EGM: An efficient generative model for unrestricted adversarial examples,” *ACM Trans. Sensor Netw.*, vol. 18, no. 4, p. 51, 2022.
- [43] B. Biggio et al., “Evasion attacks against machine learning at test time,” in *Proc. ECML/PKDD*, 2013, pp. 387–402.
- [44] J. Lu, T. Issaranon, and D. Forsyth, “SafetyNet: Detecting and rejecting adversarial examples robustly,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2017, pp. 58–66.
- [45] A. Luo, E. Li, Y. Liu, X. Kang, and Z. J. Wang, “A capsule network based approach for detection of audio spoofing attacks,” in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2021, pp. 6359–6363.
- [46] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *Proc. Int. Conf. Learn. Rep.*, 2017, pp. 1–8.
- [47] A. Liu et al., “Perceptual-sensitive GAN for generating adversarial patches,” in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1028–1035.
- [48] A. Kurakin, I. J. Goodfellow, and S. Bengio, “Adversarial examples in the physical world,” 2016, *arXiv:1607.02533*.
- [49] K. Eykholt et al., “Robust physical-world attacks on deep learning models,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1625–1634.
- [50] B. Tran, J. Li, and A. Madry, “Spectral signatures in backdoor attacks,” 2018, *arXiv:1811.00636*.
- [51] M. Du, R. Jia and D. Song, “Robust anomaly detection and backdoor attack detection via differential privacy,” 2019, *arXiv:1911.07116*.
- [52] C. Li et al., “Backdoor attack on machine learning based Android malware detectors,” *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 5, pp. 3357–3370, Sep/Oct. 2022.

- [53] B. Chen et al., “Detecting backdoor attacks on deep neural networks by activation clustering,” in *Proc. 33rd AAAI Conf. Artif. Intell. Workshop Artif. Intell. Safety (AAAI)*, 2019, pp. 1–8.
- [54] G. F. Elsayed, I. J. Goodfellow, and J. Sohl-Dickstein, “Adversarial reprogramming of neural networks,” 2018, *arXiv:1806.11146*.
- [55] R. S. S. Kumar, D. O. Brien, K. Albert, S. Vilj  en, and J. Snover, “Failure modes in machine learning systems,” 2019, *arXiv:1911.11034*.
- [56] E. Anthi, L. Williams, A. Javed, and P. Burn, “Hardening machine learning denial of service (DoS) defences against adversarial attacks in IoT smart home networks,” *Comput. Security*, vol. 108, pp. 1–20, Sep. 2021.
- [57] O. Willers, S. Sudholt, S. Raafatnia, and S. Abrecht, “Safety concerns and mitigation approaches regarding the use of deep learning in safety-critical perception tasks,” in *Proc. Comput. Safety Rel. Security (SAFECOMP Workshops)*, 2020, pp. 336–350.
- [58] T. S  mann, P. Schlicht, F. H  ger, “Strategy to increase the safety of a DNN-based perception for HAD systems,” 2020, *arXiv:2002.08935*.
- [59] G. Schwalbe, B. Knie, T. S  mann, T. Dobberphul, L. Gauerhof, and S. V. Rocco, “Structuring the safety argumentation for deep neural network based perception in automotive application,” in *Proc. Comput. Safety Rel. Security (SAFECOMP Workshops)*, 2020, pp. 383–394.
- [60] S. Burton, C. Hellert, F. H  ger, M. Mock, and A. Rohatschek, “Safety assurance of machine learning for perception functions,” in *Deep Neural Networks and Data for Automated Driving: Robustness, Uncertainty Quantification, and Insights Towards Safety*. Cham, Switzerland: Springer, 2022, pp. 335–358.
- [61] J. Ye, Y. Hu, and X. Li, “Hardware trojan in FPGA CNN accelerator,” in *Proc. IEEE 27th Asian Test Symp. (ATS)*, Oct. 2018, pp. 68–73.
- [62] Y. Nozaki, S. Takemoto, Y. Ikezaki and M. Yoshikawa, “LUT oriented hardware trojan for FPGA based AI module,” in *Proc. 6th Int. Conf. Appl. Syst. Innov. (ICASI)*, 2020, pp. 46–49, doi: [10.1109/ICASI49664.2020.9426247](https://doi.org/10.1109/ICASI49664.2020.9426247).
- [63] K. Inaba, T. Yoneda, T. Kanamoto, A. Kurokawa, and M. Imai, “Hardware trojan insertion and detection in asynchronous circuits,” in *Proc. 25th IEEE Int. Symp. Asyn. Circuits Syst. (ASYNC)*, May 2019, pp. 134–143.
- [64] T. Iwase, Y. Nozaki, M. Yoshikawa, and T. Kumaki, “Detection technique for hardware trojans using machine learning in frequency domain,” in *Proc. IEEE 4th Global Conf. Consum. Electron. (GCCE)*, Oct. 2015, pp. 185–186.
- [65] D. Fujimoto, S. Nin, Y. Hayashi, N. Miura, M. Nagata, and M. Matsumoto, “A demonstration of a HT-detection method based on impedance measurements of the wiring around ICs,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 10, pp. 1320–1324, Oct. 2018.
- [66] K. Hasegawa, M. Yanagisawa, and N. Togawa, “Empirical evaluation and optimization of hardware-trojan classification for gate-level netlists based on multi-layer neural networks,” *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, vol. E101-A, no. 12, pp. 2320–2326, Dec. 2018.
- [67] U. Guin, Z. Zhou, and A. Singh, “Robust design-for-security architecture for enabling trust in IC manufacturing and test,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 5, pp. 818–830, May 2018, doi: [10.1109/TVLSI.2018.2797019](https://doi.org/10.1109/TVLSI.2018.2797019).
- [68] M. Yasin, J. J. Rajendran, O. Sinanoglu, and R. Karri, “On improving the security of logic locking,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 9, pp. 1411–1424, Sep. 2016, doi: [10.1109/TCAD.2015.2511144](https://doi.org/10.1109/TCAD.2015.2511144).
- [69] J. Zhang, Z. Wang, and N. Verma, “In-memory computation of a machine-learning classifier in a standard 6T SRAM array,” *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, Apr. 2017.
- [70] M. E. Sinangil et al., “A 7-nm compute-in-memory SRAM macro supporting multi-bit input, weight and output and achieving 351 TOPS/W and 372.4 GOPS,” *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 188–198, Jan. 2021.
- [71] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, “XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks,” *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.
- [72] C. Yu, T. Yoo, K. T. C. Chai, T. T.-H. Kim, and B. Kim, “A 65-nm 8T SRAM compute-in-memory macro With column ADCs for processing neural networks,” *IEEE J. Solid-State Circuits*, vol. 57, no. 11, pp. 3466–3476, Nov. 2022, doi: [10.1109/JSSC.2022.3162602](https://doi.org/10.1109/JSSC.2022.3162602).
- [73] A. M. S. Tosson, S. Yu, M. H. Anis, and L. Wei, “A study of the effect of RRAM reliability soft errors on the performance of RRAM-Based neuromorphic systems,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 11, pp. 3125–3137, Nov. 2017, doi: [10.1109/TVLSI.2017.2734819](https://doi.org/10.1109/TVLSI.2017.2734819).
- [74] M. Giordano et al., “CHIMERA: A 0.92 TOPS, 2.2 TOPS/W edge AI accelerator with 2 MByte on-chip foundry resistive RAM for efficient training and inference,” in *Proc. Symp. VLSI Circuits*, 2021, pp. 1–2, doi: [10.23919/VLSICircuits52068.2021.9492347](https://doi.org/10.23919/VLSICircuits52068.2021.9492347).
- [75] A. S. Lele et al., “A heterogeneous RRAM in-memory and SRAM near-memory SoC for fused frame and event-based target identification and tracking,” *IEEE J. Solid-State Circuits*, early access, Aug. 8, 2023, doi: [10.1109/JSSC.2023.3297411](https://doi.org/10.1109/JSSC.2023.3297411).
- [76] Y. Wang et al., “Bi-LRFusion: Bi-directional LiDAR-RADAR fusion for 3D dynamic object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 13394–13403, doi: [10.1109/CVPR52729.2023.01287](https://doi.org/10.1109/CVPR52729.2023.01287).
- [77] L. Lou et al., “14.2 An early fusion complementary RADAR-LiDAR TRX in 65nm CMOS supporting gear-shifting sub-cm resolution for smart sensing and imaging,” in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2021, pp. 220–222, doi: [10.1109/ISSCC42613.2021.9365756](https://doi.org/10.1109/ISSCC42613.2021.9365756).
- [78] A. I. Mourikis and S. I. Roumeliotis, “A multi-state constraint Kalman filter for vision-aided inertial navigation,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2007, pp. 3565–3572.
- [79] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart, “Robust visual inertial odometry using a direct EKF-based approach,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2015, pp. 298–304.
- [80] J. Wang et al., “A reconfigurable matrix multiplication coprocessor with high area and energy efficiency for visual intelligent and autonomous mobile robots,” in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, 2021, pp. 1–3, doi: [10.1109/A-SSCC53895.2021.9634793](https://doi.org/10.1109/A-SSCC53895.2021.9634793).
- [81] J.-H. Yoon and A. Raychowdhury, “NeuroSLAM: A 65-nm 7.25-to-8.79-TOPS/W mixed-signal oscillator-based SLAM accelerator for edge robotics,” *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 66–78, 2021, doi: [10.1109/JSSC.2020.3028298](https://doi.org/10.1109/JSSC.2020.3028298).